

Powering Semantic Search with Cohere and Databricks Vector Search

A Practical Guide to Serving, Indexing, and Searching Baseball Scouting Reports with Cohere Embeddings in Databricks

Authors: Alexander Booth

Introduction

In this technical walkthrough, we'll demonstrate how to build a practical semantic search system using Cohere's multilingual embeddings and Databricks Vector Search. We'll walk through how to serve Cohere's embedding model via AWS Bedrock and Databricks Model Serving, use Delta Sync to maintain an up-to-date vector index, and run high-quality similarity searches.

To make it real, we'll apply this pipeline to a dataset of baseball scouting reports from [FanGraphs](#), enabling use cases like:

- "Find players with elite bat speed and raw power."
- "Find players with similar reports to Sebastian Walcott."

By the end, you'll understand why Cohere embeddings are well-suited for unstructured search and how to integrate them cleanly into production-grade pipelines on Databricks.

Purpose

The goal of this walkthrough is to show how Cohere's `embed-english-v3` model can be used to power meaningful similarity search over unstructured text, which in this case, is professional baseball scouting reports. By integrating Cohere embeddings with Databricks Model Serving and Delta Sync, we create a vector index that can be searched using both natural language queries and existing reports as semantic anchors.

This setup is designed to be:

- **Cloud-native** (via AWS Bedrock + Databricks)
- **Low-friction** (no custom pipeline maintenance)
- **Flexible** (usable for RAG, comp search, or talent profiling)

It's intended for ML engineers, data scientists, and solution architects who want a fast path from raw text to production-ready Vector Search using real-world content.

Why Cohere?

[Cohere](#) provides state-of-the-art language models optimized for retrieval, classification, and generation tasks, including high-performance embedding models. In this demo, we use the `embed-english-v3` model via **AWS Bedrock**, which is well-suited for semantic search over domain-specific text like baseball scouting reports.

Cohere's embedding model supports multiple **input types** to optimize for specific use cases:

- `search_document`: for content being indexed (like our scouting reports)
- `search_query`: for user-input queries or comparisons
- `classification` and `clustering`: for other downstream ML workflows

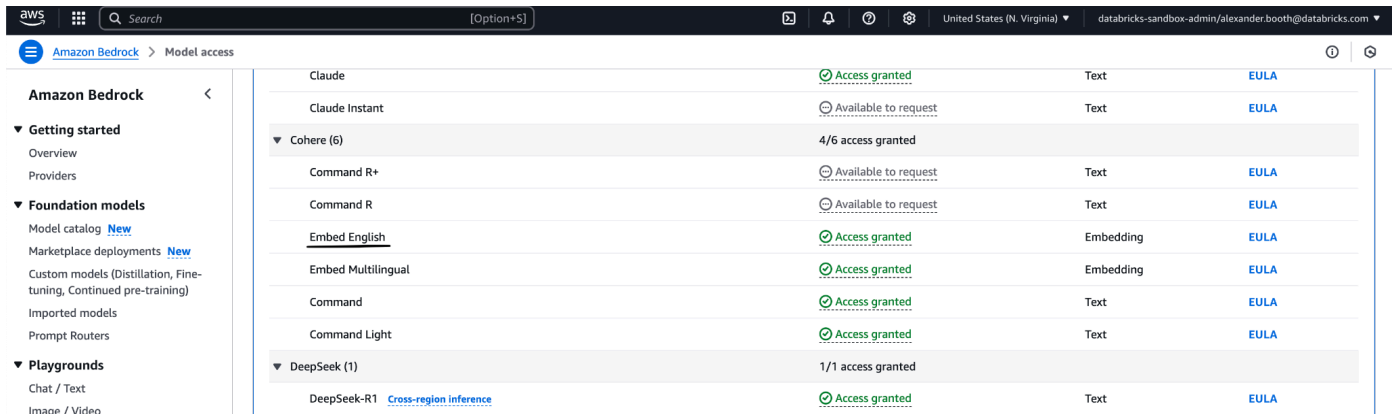
In our pipeline, we use `search_document` to embed each scouting report chunk, and `search_query` to embed user queries or reference players. These modes improve retrieval accuracy without requiring fine-tuning, making Cohere's embeddings especially effective for semantic search use cases versus models on OpenAI or HuggingFace.

Enabling Cohere

Activate Cohere in AWS Bedrock

This section walks through how to enable the Cohere `embed-english-v3` model in **AWS Bedrock**, which we'll later serve through Databricks.

- Navigate to the **AWS Management Console**
- Go to **Amazon Bedrock** → **Model access**
- Find **Cohere** under the list of providers
- Enable access to the model: `embed-english-v3`



Model	Access Status	Input Type	EULA
Claude	Access granted	Text	EULA
Claude Instant	Available to request	Text	EULA
Cohere (6)	4/6 access granted		
Command R+	Available to request	Text	EULA
Command R	Available to request	Text	EULA
<u>Embed English</u>	Access granted	Embedding	EULA
Embed Multilingual	Access granted	Embedding	EULA
Command	Access granted	Text	EULA
Command Light	Access granted	Text	EULA
DeepSeek (1)	1/1 access granted		
DeepSeek-R1 Cross-region inference	Access granted	Text	EULA

Permissions and Model Serving Setup

Once you've enabled access to Cohere in **AWS Bedrock**, you need to ensure that Databricks can invoke the model by configuring the right permissions.

Required Permissions

Depending on how you're authenticating Databricks to AWS, you'll need either:

Option 1: Access Keys (used in this demo)

Create an IAM user with Bedrock Access. Then, generate a user key and secret.

Option 2: IAM Role with Instance Profile (recommended for production)

Use the same permissions above and attach the role to your Databricks workspace.

Activate Cohere in Databricks

To use the Cohere model from within Databricks, you'll need to register it as an **external model serving endpoint**:

1. Go to **Compute** → **Serving endpoints**
2. Click **Create** → **External model**
3. Provide:
 - A name (e.g. `cohere_embed_english_v3`)
 - **AWS credentials** (user access keys/secrets or instance profile)
 - Set the **Task type** to: `embeddings`

The screenshot shows the 'Edit serving endpoint' configuration page in Databricks. The breadcrumb trail at the top is 'Serving endpoints > cohere_embed_english_v3 > Edit serving endpoint'. The page is divided into two main sections: 'Entity details' and 'Summary'.

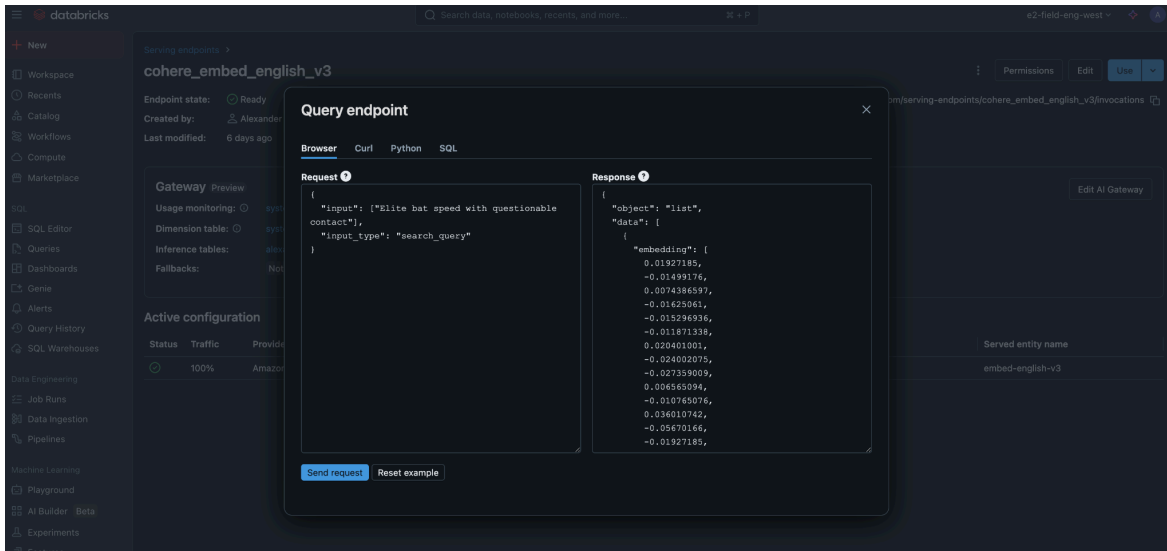
Entity details:

- Provider:** Amazon Bedrock (with a dropdown arrow and a '100' value next to it).
- Amazon Bedrock Configuration:** For more information on provider configuration, please refer to the [documentation](#).
- AWS region:** us-east-1 (with a dropdown arrow).
- Authentication method:** Chose between Instance Profile ARN or Access Keys. The 'Instance Profile ARN' radio button is selected.
- Instance profile:** Select instance profile (with a dropdown arrow).
- Bedrock provider:** Cohere (with a dropdown arrow).
- Task:** Embeddings (with a dropdown arrow).
- Provider model:** Select a model from the dropdown or enter the model name you want to use. The dropdown list is not exhaustive. If the model you want to use is not listed, enter the model name. The selected model is 'embed-english-v3' (with a dropdown arrow).

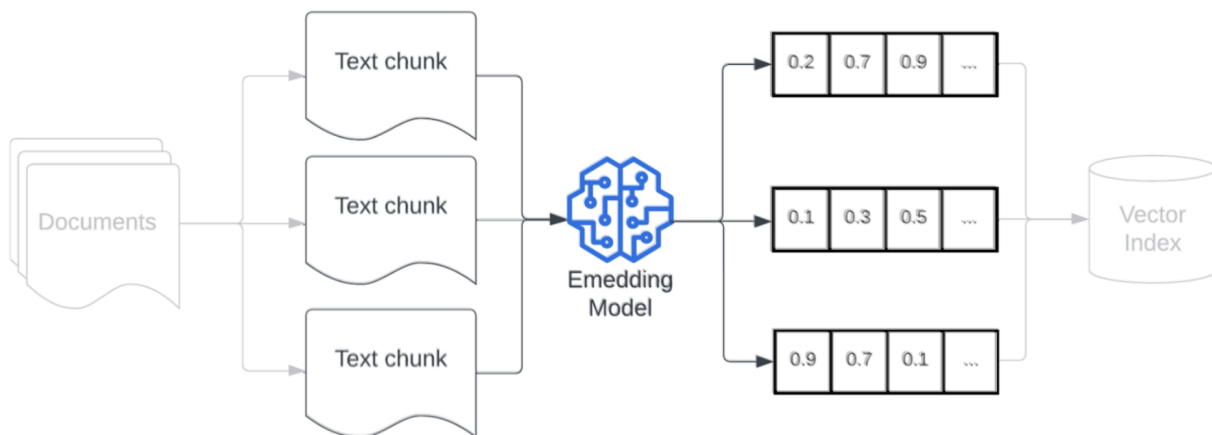
Summary:

- Served entities:** embed-english-v3 Amazon Bedrock < 100%

Test the endpoint with a sample payload. Once this endpoint is active, you can use it for embedding within Delta Sync, `vector_search()`, or direct calls. You should see embeddings returned in the response. Note that the `input_type` of "search_query" is used for retrieval tasks.



Building a Semantic Search Pipeline with Cohere and Databricks



Step 1: Load Scouting Report Data from FanGraphs

We begin by pulling prospect data directly from **FanGraphs**, which publishes detailed scouting reports across different draft boards — including preseason, midseason, and international prospects.

In this demo, we load the **preseason 2025 prospect board**, which includes:

- Player names and IDs
- Tool grades (e.g., power, speed, defense)
- Overall future value (FV)
- Written scouting summaries

01 - Load Data:

- Fetch the FanGraphs JSON API for a specific board (<https://www.fangraphs.com/api/prospects/board/data>)
- Extract and flatten the **players** section
- Combine raw text fields like **TLDR**, **Summary**, and **Ovr_Summary** into a new column
- Write the results to a Delta table

This gives us a clean, structured table of raw scouting data to work with, which acts as the foundation for chunking, embedding, and search in later steps. We also normalize the text so each player has one combined field that reflects what a scout might write on a player card.

Catalog Explorer > alexander_booth > cohere_demo >

fangraphs_mlb_scouting_reports Open in a dashboard Create

No certification set

Overview **Sample Data** Details Permissions Policies History Lineage Insights Quality

Ask Genie Preview

	llevel	mlevel	minorMasterId	playerName	playerNameRoute	UPURL	RPM
1	MLB	MLB	sa3025686	Roki Sasaki	Roki Sasaki	/players/roki-sasaki/35323/stats?position=P	
2	AAA	AAA	sa3020211	Roman Anthony	Roman Anthony	/players/roman-anthony/sa3020211/stats?position=OF	
3	MLB	MLB	sa3022882	Dylan Crews	Dylan Crews	/players/dylan-crews/33541/stats?position=OF	
4	AA	AA	sa3021069	Sebastian Walcott	Sebastian Walcott	/players/sebastian-walcott/sa3021069/stats?position=SS	
5	AAA	AAA	sa3015716	Samuel Basallo	Samuel Basallo	/players/samuel-basallo/sa3015716/stats?position=C/1B	
6	AAA	AAA	sa3017880	Andrew Painter	Andrew Painter	/players/andrew-painter/sa3017880/stats?position=P	2400 / 2500
7	MLB	MLB	sa3023011	Kristian Campbell	Kristian Campbell	/players/kristian-campbell/33644/stats?position=2B	
8	MLB	MLB	sa3019892	Dalton Rushing	Dalton Rushing	/players/dalton-rushing/31382/stats?position=C	
9	MLB	MLB	sa3018054	Jackson Jobe	Jackson Jobe	/players/jackson-jobe/30203/stats?position=P	2400 / 2900
10	AAA	AAA	sa3017795	Carson Williams	Carson Williams	/players/carson-williams/sa3017795/stats?position=SS	

Step 2: Chunking Scouting Reports for Embedding

Cohere's **embed-english-v3** model supports a maximum input length of **2048 characters** per request. However, many scouting reports, especially those with detailed summaries or multiple sections, exceed that limit.

To solve this, we chunk each report into segments of up to 2048 characters, ensuring:

- Compatibility with the model's input constraints
- Preservation of semantic coherence across chunks
- Maximum content coverage during indexing

02 - Prepare Text Chunks:

- Load the full scouting report table (`fangraphs_mlb_scouting_reports`)
- Split `Combined_Scouting_Report` into character-based chunks
- Add a `chunk_index` and builds a `primary_key` for each chunk
- Save the output as a new Delta table

Why Chunking Matters

If a report exceeds the token limit and isn't chunked, it will either:

- Fail during embedding
- Or silently truncate content, leading to poor or misleading embeddings

By chunking proactively, we ensure that every part of every report can be indexed and retrieved without losing important context for similarity search.

Step 3: Creating the Vector Store and Delta Sync Index

With the chunked scouting reports prepared, we now create a vector index using Databricks Vector Search and embed each chunk using Cohere's `embed-english-v3` model served through AWS Bedrock.

This process allows us to search semantically across all chunks, which enables use cases like phrase matching, comp search, and more.

03 - Create a Vector Search Index:

- Initialize the **Databricks Vector Search Client**
- Create a **Vector Search endpoint** (if it doesn't already exist)
- Register a **Delta Sync index** on the chunked table
- Associate the index with:
 - The chunk text column (`content_chunk`)
 - A **Cohere embedding model endpoint**
 - A **primary key** for tracking each row (`ID_Season_ChunkIndex`)
- Wait for the index to finish syncing

Key Features of Delta Sync Indexing:

- Embeddings are computed automatically using your model serving endpoint
- Syncs trigger automatically on data changes or schema updates
- The index can be queried using SQL or Python — no need to manage a vector DB

The screenshot displays the Databricks Catalog Explorer interface for a vector index named 'fangraphs_mlb_scouting_reports_index'. The interface is divided into several sections:

- Overview:** Shows the index status as 'Online', index type as 'Delta Sync', source table as 'alexander_booth.cohere_demo.fangraphs_mlb_scouting_reports_chunked', serving endpoint as 'vs_scouting_reports_demo', sync schedule as 'Triggered', and rows indexed as '1,119'.
- About this vector index:** Shows the state as 'Active', owner as 'Alexander Booth', type as 'Foreign', data source as 'Vector Index', and popularity as '----'.
- Data Ingest:** Shows the pipeline id as '0a828be0-2454-4d55-a3f5-8b3cf83f4ffe', update status as 'Completed', last processed commit as '1', and last processed timestamp as 'May 21, 2025, 10:36 AM (2 days ago)' with a 'Sync now' button.
- Embedding Model:** Shows the endpoint status as 'Ready' and the endpoint name as 'cohere_embed_english_v3'.

With this setup, we now have a live, production-ready vector store that combines Delta Lake for durability and governance, Cohere embeddings for rich semantic understanding, and Databricks Vector Search for fast, scalable similarity queries.

Step 4: Running Semantic Search Over Scouting Reports

With our vector index in place, we can now run semantic similarity searches using either:

- A **natural language query** (e.g., *"elite bat speed, struggles with offspeed pitching"*)
- An **existing player's report** (e.g., *"find comps for Sebastian Walcott"*)

Instead of relying on keyword matches, these searches use Cohere embeddings to compare meaning, which enables more intuitive and insightful exploration of player profiles.

04 - Semantic Search:

- Use the **Python SDK** to query the vector index
- Group matching chunks by player to show the most relevant scouting reports
- Join the results with the original report table to display full profiles

In this demo, we demonstrate two semantic search use cases. First, we embed a free-text query like: *"Elite bat speed but has trouble with offspeed pitches."*

And return players whose reports reflect those traits, even if the exact words aren't used.

```
# Build the full index name
FULL_INDEX_NAME = f"{{CATALOG}}.{{SCHEMA}}.{{INDEX_NAME}}"

# Retrieve the index object
index = vsc.get_index(
    endpoint_name=VECTOR_SEARCH_ENDPOINT,
    index_name=FULL_INDEX_NAME
)

QUERY = "Elite bat speed but has trouble with offspeed pitches"

final_df = search_similar_players_by_text(
    index=index,
    query_text=QUERY
)

display(final_df)
```

> [See performance \(3\)](#)

final_df: pyspark.sql.connect.dataframe.DataFrame = [ID: long, playerName: string ... 3 more fields]

[NOTICE] Using a notebook authentication token. Recommended for development only. For improved performance, please use Service Principal based authentication. To disable this message, pass disable_notice=True.

Table	ID	playerName	Season	Combined_Scouting_Report
1	2424820	Jac Caglianone	2025	TLDR: Caglianone was a two-way college player with elite power and arm strength. If his plate discipline and focus on hitting, he's going to explode. Full Report: The freakishly talented Caglianone presented a serious conundrum to teams picking atop the 2024 draft talented and tooly a college player as there has been in the last decade or so, wielding elite raw power, elite fastball velocity, but Caglianone's plate discipline and command were both so lacking that t entire profile on both sides of the ball. He slugged .738 and hit 33 homers as a sophomore, then slashed .350 in his draft year, but long levers and poor ball/strike recognition (his chase rates approached .400) caused Cags to whiff quite a bit more than his 8.2% junior year K% might otherwise indicate. His swing, aggressive hand load way up and away from his body, takes advantage of Caglianone's prodigious athletic power, but it might benefit from being toned down, not only to help stop Cags from striking out to

Finally, we embed a full scouting report and return similar players, ranked by semantic similarity. This enables "who does he remind you of?" workflows, which is a perfect fit for modern scouting and comparison analysis.

```
final_df = find_similar_to_player_report(
    index=index,
    player_name='Sebastian Walcott',
    season=2025
)

display(final_df)
```

> [See performance \(4\)](#)

final_df: pyspark.sql.connect.dataframe.DataFrame = [playerName: string, Season: integer ... 2 more fields]

[NOTICE] Using a notebook authentication token. Recommended for development only. For improved performance, please use Service Principal based authentication. To disable this message, pass disable_notice=True.

	playerName	Season	Combined_Scouting_Report
3	Cam Smith	2025	▼ TLDR: Smith has similarities to Alec Bohm when he was a fresh draftee. He might get too big to play third base, and his swing is actualized to get to all of his raw power. Smith should be an everyday player if even one of these things falls his way. Full Report: Smith is an XL third baseman with rare defensive ability for a player his size. Built like an outside linebacker at 6-foot-3 and 225 pounds or so, Smith slashed .331/.420/.594 during his career at Florida State and showed substantial improvement in strikeouts and walk peripherals as a draft-eligible sophomore. The Cubs selected him 14th overall (he was seventh on our Board) and Smith posted strong contact and power numbers after he signed. He slashed .313/.396/.609 in 134 PA, with an 84% zone contact rate (77% overall), 111 mph max exit velo, 107 mph EV90 (the big league average is 104-105), and a hard-hit rate. Those are pretty good, and they reinforced Smith's fairly aggressive draft ranking. He was then traded to

5 rows | 2.81s runtime

Refreshed 2 minutes ago

By the end of this step, we have created a powerful, LLM-ready search layer that understands the intent of a query, not just the vocabulary, unlocking richer analysis, discovery, and player comparison.

Conclusion and Next Steps

In this demo, we built a complete semantic search pipeline:

- Loaded real scouting data from **FanGraphs**
- Combined and chunked reports for embedding
- Embedded each chunk using **Cohere's embed-english-v3** model served through **Databricks Model Serving**
- Created a Delta Sync vector index for real-time, queryable search
- Performed semantic search using both natural language phrases and full reports

This approach gives us a faster and more meaningful way to search unstructured text. It aligns more closely with how scouts and analysts think about players, rather than relying on simple keyword matches.

Coming Soon: A RAG-Based Scouting Assistant

In a future demo, we will extend this work using **Retrieval-Augmented Generation (RAG)** to build a prototype assistant that can:

- Answer open-ended questions using real scouting reports as context
- Summarize groups of similar players
- Let users explore comps and player traits in a conversational format

This will combine LLMs with vector search to make scouting reports not just searchable but fully interactive.

Stay tuned.

References

- **Cohere Embedding Models**
[Introduction to Embeddings at Cohere](#)
- **AWS Bedrock: Use Cohere Models**
<https://docs.aws.amazon.com/bedrock/latest/userguide/model-access.html>

- **Databricks External Model Serving**
[Create custom model serving endpoints | Databricks Documentation](#)
- **Databricks Vector Search Overview**
[Mosaic AI Vector Search | Databricks Documentation](#)
- **Create Vector Search Index with Delta Sync**
[How to create and query a vector search index | Databricks Documentation](#)
- **FanGraphs Prospect Boards**
[The Board | FanGraphs Baseball](#)
- **Build an Unstructured Data Pipeline for RAG**
[Build an unstructured data pipeline for RAG | Databricks Documentation](#)